CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

DISCIPLINA DESENVOLVIMENTO WEB

GRUPO 1

Mauro César Maia Reis — 04214707

Eduardo Gabriel Soares Ferreira — 04209877

Patrick de Oliveira Almeida — 26075797

Caio Renato Nascimento Donati — 04214983

Luís Mário Marques Costa — 04214724

REACT NO DESENVOLVIMENTO WEB

Fundamentos e análise comparativa com Angular e Vue

Trabalho acadêmico de pesquisa

Mauro César Maia Reis — Matrícula 04214707
Eduardo Gabriel Soares Ferreira — Matrícula 04209877
Patrick de Oliveira Almeida — Matrícula 26075797
Caio Renato Nascimento Donati — Matrícula 04214983
Luís Mário Marques Costa — Matrícula 04214724

REACT NO DESENVOLVIMENTO WEB

Fundamentos e análise comparativa com Angular e Vue

Trabalho acadêmico de pesquisa apresentado à disciplina Desenvolvimento Web, do curso de Análise e Desenvolvimento de Sistemas, pelo Grupo 1, com foco na análise do React e na especificação de uma aplicação prática.

RESUMO

Este trabalho analisa o React como tecnologia para desenvolvimento de interfaces web, considerando sua origem, fundamentos e implicações de uso. A pesquisa bibliográfica e documental prioriza fontes oficiais e compara React com Angular e Vue quanto à aprendizagem, desempenho, escalabilidade, manutenção, popularidade, oportunidades profissionais e adequação a projetos. São examinados componentes, props, estado, Hooks, atualização da interface, roteamento, integração com APIs e ferramentas de desenvolvimento. A análise identifica vantagens na composição e na descrição declarativa, assim como desafios na definição de arquitetura e no uso consistente do estado. Indicadores de adoção são interpretados como dados amostrais, sem equivalência automática com vagas ou participação de mercado. Como aplicação prática, especifica-se um gerenciador acadêmico de tarefas, com requisitos, organização de componentes, critérios de validação e responsabilidades propostas. A implementação e as evidências de execução serão produzidas pela equipe. Conclui-se que React é uma escolha adequada para demonstrar interfaces interativas, desde que sua flexibilidade seja acompanhada de convenções e verificação de comportamento.

Palavras-chave: React. Desenvolvimento web. Componentização. Estado. Frameworks front-end.

1 INTRODUÇÃO

O desenvolvimento de interfaces web envolve decisões que afetam a experiência do usuário e a capacidade de manter o software. Uma tela de cadastro precisa combinar validação, atualização de dados, navegação e mensagens de erro. Quando esses comportamentos são distribuídos em manipulações isoladas do documento HTML, torna-se difícil compreender como uma alteração produz seus efeitos. Tecnologias baseadas em componentes oferecem uma forma de organizar esse problema.

Este trabalho analisa o React, tema do Grupo 1, como tecnologia de construção de interfaces. Embora o enunciado o reúna à categoria dos frameworks front-end, sua documentação o define como biblioteca de interfaces. Essa distinção orienta a pesquisa: React fornece o modelo de composição e atualização da interface; outras necessidades podem ser atendidas por bibliotecas ou frameworks construídos sobre ele (REACT, Quick Start; Creating a React App).

O objetivo geral é avaliar fundamentos, recursos, vantagens e limites do React, relacionando-os a decisões de desenvolvimento. Os objetivos específicos são contextualizar sua origem, explicar componentização e estado, examinar integração com APIs e roteamento, discutir adoção profissional e comparar React com Angular e Vue. Como desdobramento, apresenta-se uma especificação de aplicação que permita à equipe demonstrar os conceitos estudados.

A metodologia é uma pesquisa bibliográfica e documental de natureza qualitativa. Foram priorizadas documentações oficiais e publicações dos responsáveis pelos projetos. Informações de adoção são tratadas como indicadores delimitados pela amostra consultada. A comparação não utiliza experimentos de desempenho e, por isso, não apresenta um ranking de velocidade nem resultados numéricos de execução.

A análise indica que React é apropriado para interfaces interativas quando a equipe estabelece convenções de arquitetura, tratamento de dados e qualidade. Sua flexibilidade exige escolhas explícitas. A aplicação prática é apresentada como projeto de um gerenciador acadêmico de tarefas. Implementação, comprovação dos testes, repositório público e publicação serão realizados pela equipe. O relatório distingue essa proposta de resultados efetivamente medidos.

2 CONTEXTUALIZAÇÃO HISTÓRICA

React surgiu no Facebook, hoje Meta, associado ao trabalho de Jordan Walke e à necessidade de organizar interfaces com atualizações frequentes. Sua disponibilização pública como projeto de código aberto ocorreu em 2013. Em publicação daquele período, Pete Hunt explica que a proposta era construir interfaces componíveis e reutilizáveis, reduzindo a necessidade de sincronizar manualmente os dados com cada alteração do DOM, a representação da página mantida pelo navegador (HUNT, Why did we build React?).

A motivação pode ser compreendida pelo exemplo de uma lista de mensagens. Uma nova mensagem pode alterar o contador, a posição dos itens e os indicadores de leitura. Manter cada elemento por comandos independentes cria dependências difíceis de acompanhar. O modelo declarativo permite descrever o resultado esperado a partir dos dados e delegar ao mecanismo da biblioteca a atualização da interface. Isso desloca parte da complexidade para a modelagem do estado.

Um marco posterior foi React 16.8, lançado em 2019, que introduziu Hooks. Esses recursos possibilitaram utilizar estado e outras capacidades do React em componentes de função, sem exigir classes nesses casos. A mudança favoreceu a extração de comportamentos reutilizáveis em Hooks personalizados. A compatibilidade com componentes existentes permitiu adoção gradual, sem obrigar a reescrita imediata das aplicações (ABRAMOV, React v16.8).

React 18, lançado em 2022, acrescentou bases para renderização concorrente e recursos como transições, além de ampliar possibilidades de Suspense no servidor. Concorrência, nesse contexto, não significa executar automaticamente toda a interface em múltiplas threads: envolve organizar e interromper trabalho de renderização para priorizar interações. A utilidade depende de como esses recursos são empregados (REACT, React v18.0).

A evolução do projeto ampliou possibilidades sem eliminar a necessidade de fundamentos de JavaScript, HTML e CSS. A documentação e o desenvolvimento contam com colaboradores de diferentes organizações e comunidades. A empresa de origem não deve ser confundida com a composição permanente de toda a manutenção (REACT, Meet the Team). Para o projeto acadêmico, interessa selecionar recursos estáveis e justificar seu uso, em vez de adotar funcionalidades apenas por serem recentes.

3 FUNDAMENTAÇÃO TEÓRICA

Uma biblioteca oferece recursos que o desenvolvedor integra à aplicação. Um framework costuma definir uma organização mais abrangente e coordenar partes importantes de sua execução. A fronteira nem sempre é rígida, mas ajuda a compreender o papel do React: seu núcleo se concentra na descrição de interfaces. Uma solução completa pode acrescentar roteamento, carregamento de dados e estratégias de renderização no servidor (REACT, Creating a React App).

O paradigma declarativo descreve o que deve aparecer para determinado conjunto de dados. Em um gerenciador de tarefas, a mensagem de lista vazia decorre de uma coleção sem itens; não precisa ser mantida por comandos independentes para esconder e mostrar elementos. Essa abordagem facilita raciocinar sobre carregamento, sucesso e erro. Contudo, a qualidade depende de representar esses estados de maneira consistente e evitar combinações contraditórias.

A componentização decompõe a interface em unidades com responsabilidades identificáveis. Um componente pode representar um botão, um formulário ou uma seção da página. Props são entradas recebidas do componente superior; estado representa a memória necessária ao comportamento local. A composição combina essas unidades em estruturas maiores. A divisão deve seguir responsabilidades e mudanças esperadas, evitando componentes extensos ou fragmentação sem benefício (REACT, Describing the UI).

JSX é uma extensão de sintaxe utilizada para escrever descrições da interface próximas da lógica JavaScript. Sua aparência semelhante a HTML não significa que seja uma página HTML independente. Expressões entre chaves permitem inserir valores e condições; o processo de construção transforma essa sintaxe para execução. O aprendizado exige compreender funções, objetos, arrays e eventos, além das diferenças entre propriedades JSX e atributos HTML (REACT, Quick Start).

React favorece um fluxo de dados descendente: o componente superior fornece valores e funções aos descendentes. Quando um usuário altera uma tarefa, o descendente pode chamar uma função recebida para solicitar a mudança no estado responsável. Assim, a propriedade do dado permanece identificável. Props não devem ser alteradas pelo componente que as recebe. Na aplicação proposta, a coleção de tarefas terá uma fonte de verdade, da qual serão derivados a lista filtrada e os indicadores.

4 ARQUITETURA E ATUALIZAÇÃO DA INTERFACE

A arquitetura de uma aplicação React pode ser entendida como uma árvore de componentes. O componente raiz organiza páginas ou áreas; componentes internos descrevem partes menores. A arquitetura do produto também precisa definir limites entre interface, regras e acesso a dados. React não determina sozinho como nomear pastas, validar entradas ou estabelecer contratos com um servidor. Essas convenções devem ser acordadas pela equipe.

O processo de atualização envolve um gatilho, a renderização e a confirmação das alterações. Renderizar significa chamar componentes para calcular a descrição da interface. Na etapa de commit, as alterações necessárias são aplicadas ao DOM. Uma nova renderização não implica reconstruir toda a página: se uma parte permanece equivalente, pode não haver mudança correspondente no DOM (REACT, Render and Commit).

A representação intermediária da interface é frequentemente chamada de DOM virtual. A reconciliação relaciona a descrição anterior com a nova para decidir atualizações. Em listas, identificadores estáveis ajudam a associar cada componente à entidade correta. Usar a posição como chave em uma coleção que permite exclusão e reorganização pode associar estado local ao item errado. No projeto proposto, o identificador da tarefa deve permanecer estável.

A renderização deve ser pura: calcular a interface não deve modificar objetos externos nem iniciar ações com consequências imprevisíveis. Ao atualizar uma coleção, produzir uma nova estrutura torna a mudança explícita. Concluir uma tarefa exige uma coleção com o item atualizado, preservando os demais. Essa organização reduz efeitos indiretos que dificultam investigar defeitos (REACT, Render and Commit; Managing State).

Efeitos servem à sincronização com sistemas externos, como assinaturas de eventos ou armazenamento do navegador. Não devem ser a solução automática para todo cálculo. Se o número de tarefas concluídas pode ser obtido da coleção, mantê-lo como estado separado cria redundância. Operações externas exigem atenção às dependências e à limpeza, evitando assinaturas repetidas e respostas antigas sobrescrevendo dados recentes (REACT, Synchronizing with Effects).

Separar componentes, funções de domínio e persistência permite substituir armazenamento local por API sem redesenhar toda a interface. Essa separação é uma decisão de engenharia proposta para o projeto, e não uma garantia automática da biblioteca.

5 RECURSOS E FUNCIONALIDADES

O gerenciamento de estado começa pela identificação do proprietário de cada informação. O texto ainda não enviado de um formulário pode permanecer no componente de edição. A coleção consultada pela lista e pelo painel deve ficar no ancestral comum responsável por ambos. Esse movimento é conhecido como elevação de estado. Evitar cópias independentes do mesmo dado diminui divergências entre áreas da tela (REACT, Managing State).

useState atende mudanças locais simples. useReducer pode organizar transições mais complexas em ações explícitas, como adicionar, editar, concluir e remover. Context disponibiliza dados a componentes distantes sem repassá-los por todas as camadas intermediárias. Seu uso deve refletir uma necessidade concreta: concentrar informações indiscriminadamente em um contexto amplo dificulta acompanhar dependências e atualizações.

A reatividade do React se manifesta quando atualizações de estado provocam novos cálculos da interface. Alterar uma variável comum não substitui a atualização de estado. Eventos conectam ações do usuário às mudanças. Em formulários controlados, o valor exibido deriva do estado e o evento de alteração informa o novo conteúdo, facilitando validação e mensagens de orientação (REACT, State A Component's Memory).

Roteamento não integra o núcleo do React. React Router é uma opção para associar endereços a telas, trabalhar com parâmetros e organizar navegação. No gerenciador proposto, a rota principal exibiria a lista e uma rota de detalhe identificaria a tarefa. A URL deve possibilitar acesso direto ao recurso; a hospedagem precisa ser configurada conforme a estratégia de rotas (REACT ROUTER, Installation).

A escolha de bibliotecas complementares deve considerar tamanho do projeto e manutenção. Uma aplicação pequena pode funcionar com recursos do React e do navegador. Estado compartilhado complexo, formulários extensos e sincronização remota podem justificar soluções especializadas, desde que responsabilidades e versões sejam registradas. Dependências sem finalidade concreta aumentam o esforço de compreensão.

Na aplicação, eventos atualizam o estado, a lista e os indicadores. A equipe deverá explicar esse percurso no código.

6 INTEGRAÇÃO COM APIS E FERRAMENTAS

A comunicação com APIs permite consumir e enviar dados externos. Em uma arquitetura HTTP, uma camada de acesso concentra endereços, serialização e tratamento de respostas. A tela precisa representar carregamento, erro, ausência de resultados e sucesso. Essa modelagem é uma decisão do projeto: separar comunicação da apresentação evita repetir regras de rede em vários componentes.

Na aplicação proposta, a primeira implementação poderá utilizar armazenamento local e uma interface de serviço que permita substituição posterior por API. Essa escolha torna o escopo viável e evita depender de um serviço externo para avaliar o fluxo principal. Persistência local não oferece sincronização entre dispositivos, controle de acesso nem garantia de disponibilidade. Essas limitações deverão constar da documentação do código.

Em consultas remotas, é preciso impedir que respostas desatualizadas substituam as mais recentes. A documentação orienta considerar limpeza ou descarte de resultados em efeitos com operações externas. Também será necessário tratar falhas e permitir nova tentativa. Gravações motivadas por um clique devem estar vinculadas ao evento correspondente, evitando dispará-las apenas porque o componente renderizou (REACT, Synchronizing with Effects).

Vite é uma ferramenta de desenvolvimento e construção que pode iniciar um projeto React e gerar arquivos de produção. Seu servidor auxilia o ciclo de edição e atualização. Uma configuração simples favorece o aprendizado. A documentação de React recomenda considerar frameworks para novos produtos e admite ferramentas como Vite para aprendizado ou necessidades específicas (VITE, Getting Started; REACT, Build a React app from Scratch).

React Developer Tools permite inspecionar componentes, props e estado e investigar problemas de desempenho. Esse apoio deve ser combinado com console e ferramentas de rede. A inspeção ajuda a responder perguntas concretas: qual componente armazena a tarefa, por que uma área renderizou e qual requisição falhou (REACT, React Developer Tools).

O fluxo de qualidade proposto inclui revisão, análise estática, testes de comportamento e verificação do build. Segredos não devem ser inseridos no código entregue ao navegador. Uma futura API com dados privados exigirá validação e autorização no servidor; esconder um botão não constitui controle de acesso. Essas decisões precisam acompanhar qualquer expansão do escopo.

7 APLICAÇÕES NO MERCADO

A origem do React em produtos do Facebook ajuda a explicar sua associação com interfaces dinâmicas. A publicação histórica de Hunt documenta seu uso no Facebook e na aplicação web do Instagram. Esses registros demonstram adoção em produtos reais, mas devem ser interpretados em seu contexto: não descrevem, por si, a arquitetura integral atual dessas plataformas (HUNT, Why did we build React?).

Para examinar adoção, utiliza-se o recorte de tecnologias web da pesquisa Stack Overflow Developer Survey 2026. Entre os 10.281 respondentes desse recorte, React aparece com 41,5%, Vue.js com 17,2% e Angular com 16,0%. As opções admitem múltiplas respostas; portanto, os percentuais não são parcelas exclusivas de mercado. A pesquisa mede uso autodeclarado na amostra, não número de sistemas implantados ou vagas disponíveis (STACK OVERFLOW, Web frameworks and technologies).

O indicador sugere ampla presença de React entre os participantes e justifica seu estudo profissional. Não permite concluir que existam oportunidades na mesma proporção em Belém, em determinada empresa ou faixa salarial. Uma investigação local exigiria coletar anúncios, definir região e período, eliminar duplicidades e separar exigências obrigatórias das desejáveis. Esse levantamento não foi realizado.

A comunidade participa por documentação, discussões e contribuições. A organização pública do projeto reúne profissionais de diferentes empresas e grupos de trabalho. Isso amplia caminhos de formação, mas exige selecionar materiais compatíveis com a abordagem adotada. Exemplos antigos baseados em classes podem ser úteis em manutenção, embora não devam ser a única referência de um novo projeto de componentes de função (REACT, Meet the Team; ABRAMOV, React v16.8).

Como perspectiva técnica, a documentação aponta integração com frameworks que coordenam navegação, carregamento e renderização. Executar no cliente, renderizar no servidor ou gerar conteúdo estático são decisões relacionadas a acesso, distribuição e atualização. Essas possibilidades exigem distinguir conhecimentos do núcleo da biblioteca daqueles específicos de ferramentas adicionais (REACT, Creating a React App).

Para a formação em ADS, a implicação é desenvolver competências transferíveis: modelagem de componentes, contratos de dados, testes, acessibilidade e leitura de documentação. Conhecer React pode ampliar a participação em projetos, mas o desempenho profissional depende também de resolver problemas e justificar decisões.

8 VANTAGENS E LIMITAÇÕES

8.1 Vantagens

A primeira vantagem é a reutilização por componentes. Um mesmo campo pode atender diferentes formulários, reduzindo divergências. A segunda é a descrição declarativa: representar estados visuais facilita revisar carregamento, erro e sucesso. Esses benefícios dependem de componentes com limites bem definidos (REACT, Describing the UI).

A terceira vantagem é o fluxo rastreável de dados. Definir quem possui a informação e como solicitações de mudança circulam favorece a depuração. A quarta é a flexibilidade de integração: a equipe pode escolher soluções compatíveis com o escopo. A quinta é separar lógica reutilizável da apresentação, favorecendo consistência entre telas (REACT, Thinking in React; Managing State).

Uma sexta vantagem é o apoio de ferramentas específicas. Inspecionar a árvore e o estado facilita verificar hipóteses sobre defeitos. No projeto proposto, seria possível observar a relação entre envio do formulário, mudança da coleção e atualização dos indicadores (REACT, React Developer Tools).

8.2 Limitações e desafios

O primeiro desafio é a quantidade de decisões externas ao núcleo. Roteamento, persistência e convenções arquiteturais precisam ser escolhidos. A flexibilidade pode produzir configurações inconsistentes sem critérios compartilhados. Propõe-se começar com poucas dependências e documentar cada responsabilidade.

O segundo desafio é o modelo mental de estado e efeitos. Dados duplicados, dependências incorretas e alterações diretas de objetos podem gerar erros. Revisões devem verificar a fonte de verdade e a necessidade de cada efeito. O terceiro é o desempenho: reconciliação não elimina custo de JavaScript, requisições ou listas extensas. Otimização precisa responder a medições.

O quarto desafio é a estratégia de entrega para conteúdo público. Renderização apenas no cliente pode exigir cuidados adicionais com carregamento e indexação. Frameworks com estratégias de servidor podem ajudar, mas acrescentam responsabilidades. A adequação depende do projeto e não pode ser deduzida somente da popularidade.

9 ESTUDO COMPARATIVO

A comparação considera React, Angular e Vue, sem tratar Next.js como concorrente independente de React. Angular é um framework mantido por uma equipe do Google, com plataforma integrada. Vue oferece adoção progressiva. React concentra-se em interfaces componíveis, com escolhas complementares definidas pelo projeto (ANGULAR, What is Angular?; VUE.JS, Introduction).

A aprendizagem depende da experiência: React exige JavaScript, JSX e estado; Angular requer domínio de suas convenções; Vue permite aproximação por templates. Não foram medidos tempos de aprendizagem.

Critério	**React**	**Angular**	**Vue**
Aprendizagem	JSX e estado	Plataforma integrada	Templates e progressão
Desempenho	Depende da implementação	Depende da implementação	Depende da implementação
Escalabilidade	Convenções da equipe	Padrões integrados	Organização progressiva
Manutenção	Contratos e dependências	Convenções comuns	Coerência entre APIs
Popularidade[1]	41,5%	16,0%	17,2%
Oportunidades	Ampla adoção na amostra	Depende do empregador	Depende do contexto
Adequação	Interfaces com autonomia	Padronização de sistemas	Adoção gradual

Fonte: elaboração própria com base nas documentações e na pesquisa Stack Overflow 2026. Percentuais de uso autodeclarado; não representam distribuição de vagas.

Não foram executados benchmarks. As avaliações são qualitativas e não estabelecem superioridade universal.

10 ANÁLISE CRÍTICA DA COMPARAÇÃO

Escalabilidade envolve crescimento de usuários, equipe e código. A biblioteca da interface não resolve capacidade do servidor ou banco de dados. No front-end, o problema inclui distribuir responsabilidades, evitar dependências circulares e permitir mudanças seguras. React oferece liberdade para construir essa organização, mas requer acordos sobre estado, serviços e componentes.

Angular reúne recursos e convenções que reduzem parte das escolhas iniciais. Seu roteador oficial integra navegação, e Signals acompanha dependências de estado. Para organizações que desejam uniformidade entre equipes, esse conjunto pode ser vantajoso. O custo é aprender e manter coerência com uma plataforma mais abrangente (ANGULAR, Angular Routing; Angular Signals).

Vue combina renderização declarativa e reatividade, com Options API e Composition API como estilos de organização. Isso permite escolher uma abordagem compatível com experiência e complexidade. A existência de alternativas pede convenções internas para evitar padrões divergentes. A documentação apresenta bibliotecas oficiais como Vue Router e Pinia para compor aplicações maiores (VUE.JS, Introduction; Composition API FAQ).

Em manutenibilidade, a diferença principal não é a presença de componentes, pois as três tecnologias trabalham com essa ideia. Muda a forma de organizar atualização, dados e ferramentas. Um projeto React com contratos e testes pode ser mais fácil de manter que outro sem essas práticas; o raciocínio vale também para Angular e Vue. Popularidade não substitui qualidade de arquitetura.

Para um painel de tarefas, as três alternativas podem atender. React é justificável neste grupo porque permite demonstrar componentes, estado, eventos e composição em escopo pequeno. Se o contexto exigisse convenções institucionais uniformes, Angular mereceria atenção. Para introduzir interatividade gradualmente em páginas existentes, Vue poderia ser conveniente. São recomendações condicionais, não resultados de experimento.

A decisão final deve registrar experiência da equipe, requisitos, integrações e manutenção. Recomenda-se uma configuração React simples para a aplicação acadêmica, evitando recursos de servidor ou bibliotecas de estado sem demanda. O projeto deve demonstrar consistência nos fundamentos antes de ampliar suas ferramentas. Assim, a comparação orienta uma escolha argumentada e evita converter preferências em afirmações universais.

11 APLICAÇÃO PRÁTICA PROPOSTA

11.1 Objetivo e requisitos

Propõe-se o Organiza Estudos, gerenciador acadêmico de tarefas em React. O usuário poderá registrar atividades, informar disciplina e prioridade, editar, marcar conclusão, excluir e filtrar. Um painel exibirá total, pendentes e concluídas. O escopo não inclui autenticação, colaboração simultânea ou dados sensíveis. Esta seção especifica a implementação a ser realizada pela equipe; não relata uma aplicação já executada.

Os requisitos funcionais são cadastrar título obrigatório, listar registros, editar dados, alternar situação, excluir com confirmação e combinar filtros de texto e situação. O armazenamento local deverá preservar registros no mesmo navegador. Requisitos de qualidade incluem uso por teclado, campos identificados por rótulos, mensagens compreensíveis, adaptação a telas pequenas e tratamento de falhas na persistência.

11.2 Arquitetura e fluxo

App coordenará o estado compartilhado. TaskForm receberá valores e uma função de envio; TaskList receberá a coleção filtrada; TaskItem apresentará um registro e suas ações; TaskFilters controlará critérios; TaskSummary mostrará indicadores. Validação ficará separada da apresentação. Um serviço concentrará leitura e gravação para permitir substituição futura.

Cada tarefa terá id estável, título, disciplina, prioridade e situação. O título será normalizado e validado antes de adicionar ou editar. Lista filtrada e contagens serão calculadas a partir da coleção principal. Ao cadastrar uma atividade, a lista deverá exibi-la e o total aumentar em uma unidade; concluí-la atualizará situação e indicadores sem alterar o identificador.

A interface seguirá o percurso de preencher, validar, salvar e confirmar. Em erro de armazenamento, deverá informar a falha e preservar a sessão quando possível. A decomposição aplica ao domínio acadêmico a orientação de identificar componentes e estado mínimo (REACT, Thinking in React).

A aplicação permitirá demonstrar props no envio de dados, eventos nas ações, estado na coleção e efeitos na persistência externa à interface. Rotas podem separar lista e detalhe. A implementação deverá manter o escopo acordado para que cada funcionalidade seja verificável e documentada.

12 VALIDAÇÃO E ENTREGA DA APLICAÇÃO

O plano abaixo deverá ser executado após a implementação. Não há resultados de testes ou métricas a declarar. A equipe registrará cenário, resultado esperado, resultado obtido e evidência, corrigindo falhas antes da publicação. O objetivo é comprovar comportamentos observáveis.

Caso	Resultado esperado
Título vazio	Bloquear envio e orientar preenchimento
Cadastrar e editar	Exibir dados e preservar identificador
Concluir e filtrar	Atualizar situação, lista e indicadores
Excluir	Remover somente o item confirmado
Recarregar	Recuperar dados locais válidos
Dados inválidos	Informar falha sem impedir a abertura
Teclado e tela pequena	Permitir ações e leitura do conteúdo

A divisão sugerida é: Mauro César Maia Reis coordena relatório e fundamentos; Eduardo Gabriel Soares Ferreira organiza componentes e apresentação visual; Patrick de Oliveira Almeida trabalha estado e filtros; Caio Renato Nascimento Donati organiza persistência e testes; Luís Mário Marques Costa organiza documentação e publicação. São responsabilidades propostas, a confirmar, e não registro de atividades concluídas.

Repositório GitHub público: [INSERIR LINK REAL]. Aplicação publicada, quando disponível: [INSERIR LINK REAL]. Código-fonte, README e evidências: [INCLUIR NO REPOSITÓRIO]. A equipe deverá preencher esses campos e revisar a descrição conforme o produto final.

O README deverá informar requisitos de ambiente, versões utilizadas, instalação, comando de desenvolvimento, testes configurados e geração do build. Em um projeto Vite padrão, npm install, npm run dev e npm run build são comandos usuais, a confirmar no package.json real. Incluir limitações da persistência, estrutura de pastas e acesso. As instruções definitivas serão escritas pela equipe a partir do código final.

13 CONSIDERAÇÕES FINAIS

React oferece um modelo consistente para organizar interfaces a partir de componentes e mudanças de estado. Sua origem está ligada ao desafio de manter telas dinâmicas, e sua evolução ampliou formas de reutilizar comportamentos e coordenar atualizações. O benefício central está em descrever a interface a partir dos dados e explicitar responsabilidades.

A tecnologia não elimina decisões de engenharia. Estado duplicado, efeitos desnecessários, dependências pouco justificadas e ausência de testes prejudicam manutenção e desempenho. A flexibilidade exige padrões proporcionais ao projeto. Em uma aplicação acadêmica pequena, simplicidade favorece domínio dos fundamentos; em produtos maiores, carregamento, navegação e entrega exigem avaliação adicional.

A comparação com Angular e Vue identificou alternativas viáveis, com diferentes formas de distribuir escolhas entre ferramenta e equipe. Angular oferece plataforma integrada; Vue permite adoção progressiva; React prioriza o núcleo de interfaces e complementos. Os indicadores consultados sustentam sua relevância, mas não autorizam conclusões automáticas sobre salários, vagas locais ou superioridade técnica universal.

O Organiza Estudos traduz a análise em proposta verificável. Cadastro, edição, filtros, conclusão e persistência permitem observar relações entre componentes, eventos e estado. Os casos de teste orientam a comprovação funcional, enquanto a separação entre interface e armazenamento prepara a evolução. As decisões foram dimensionadas para demonstrar os conceitos sem infraestrutura desnecessária.

Este relatório reúne fundamentação, análise crítica e especificação prática em uma sequência que orienta a execução do grupo. Código-fonte, validação efetiva, instruções definitivas e links reais deverão ser produzidos e incorporados pela equipe. As conclusões teóricas permanecem distinguíveis da comprovação prática exigida para a entrega completa.

Conclui-se que React atende ao propósito formativo quando utilizado com responsabilidade sobre dados, componentes e qualidade. A seleção de uma tecnologia deve decorrer de necessidades concretas e da capacidade da equipe sustentá-la. A continuidade é implementar o escopo, executar a validação e atualizar o relatório com evidências que permitam reproduzir o funcionamento.

REFERÊNCIAS

ABRAMOV, Dan. React v16.8 The One With Hooks. 2019. Disponível em: https://legacy.reactjs.org/blog/2019/02/06/react-v16.8.0.html.

ANGULAR. What is Angular? Documentação oficial. Disponível em: https://angular.dev/overview.

ANGULAR. Angular Routing. Documentação oficial. Disponível em: https://angular.dev/guide/routing.

ANGULAR. Angular Signals. Documentação oficial. Disponível em: https://angular.dev/guide/signals.

HUNT, Pete. Why did we build React? 2013. Disponível em: https://legacy.reactjs.org/blog/2013/06/05/why-react.html.

REACT. Quick Start. Documentação oficial. Disponível em: https://react.dev/learn.

REACT. Describing the UI. Documentação oficial. Disponível em: https://react.dev/learn/describing-the-ui.

REACT. Render and Commit. Documentação oficial. Disponível em: https://react.dev/learn/render-and-commit.

REACT. Managing State. Documentação oficial. Disponível em: https://react.dev/learn/managing-state.

REACT. State A Component's Memory. Documentação oficial. Disponível em: https://react.dev/learn/state-a-components-memory.

REACT. Synchronizing with Effects. Documentação oficial. Disponível em: https://react.dev/learn/synchronizing-with-effects.

REFERÊNCIAS CONTINUAÇÃO

REACT. Thinking in React. Documentação oficial. Disponível em: https://react.dev/learn/thinking-in-react.

REACT. Creating a React App. Documentação oficial. Disponível em: https://react.dev/learn/creating-a-react-app.

REACT. Build a React app from Scratch. Documentação oficial. Disponível em: https://react.dev/learn/build-a-react-app-from-scratch.

REACT. React v18.0. 2022. Disponível em: https://react.dev/blog/2022/03/29/react-v18.

REACT. Meet the Team. Documentação oficial. Disponível em: https://react.dev/community/team.

REACT. React Developer Tools. Documentação oficial. Disponível em: https://react.dev/learn/react-developer-tools.

REACT ROUTER. Installation. Modo declarativo. Disponível em: https://reactrouter.com/start/declarative/installation.

STACK OVERFLOW. Web frameworks and technologies. Developer Survey 2026. Recorte All Respondents, n = 10.281. Disponível em: https://survey.stackoverflow.co/2026/technology/data/web-framework.

VITE. Getting Started. Documentação oficial. Disponível em: https://vite.dev/guide/.

VUE.JS. Introduction. Documentação oficial. Disponível em: https://vuejs.org/guide/introduction.

VUE.JS. Composition API FAQ. Documentação oficial. Disponível em: https://vuejs.org/guide/extras/composition-api-faq.html.